

گزارش سوال ۳ عملی - تمرین ۷ - سیستم عامل

Linked List

این تمرین را بر روی Debian 13 و به کمک perf و flamegraph انجام می‌دهیم.

ابتدا سایز fs را به ۱۲۸ مگابایت تغییر داده و دستور stressTest را پیاده‌سازی می‌کنیم:

```
void stress_test() {
    printf("--- Starting Stress Test (Linked List Baseline) ---\n");

    // create new fs (fully destroys any existing fs with the same name)
    if (!create_fs("stress_test.bin")) {
        printf("Error: Could not create test file.\n");
        return;
    }

    const int num_files = 2000;
    const int num_operations = 10000;
    char filenames[num_files][16];

    printf("num_files: %d\n", num_files);
    printf("num_operations: %d\n", num_operations);

    // create initial files
    for (int i = 0; i < num_files; i++) {
        snprintf(filenames[i], sizeof(filenames[i]), "file_%d", i);
        create_file(filenames[i]);
    }

    // start timers (Linux / WSL2)
    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);
```

```

bagheri-timarchi@debian:~/Desktop/OS-HW7-P-Q3/7-initial$ sudo apt update
sudo apt install linux-perf git
Hit:1 https://dl.google.com/linux/chrome/deb stable InRelease
Hit:2 http://deb.debian.org/debian trixie InRelease
Hit:3 http://deb.debian.org/debian-security trixie-security InRelease
Err:4 https://download.docker.com/linux/debian trixie InRelease
  403 Forbidden [IP: 13.35.182.118 443]
Hit:5 http://deb.debian.org/debian trixie-updates InRelease
Hit:6 https://download.virtualbox.org/virtualbox/debian trixie InRelease
Error: Failed to fetch https://download.docker.com/linux/debian/dists/trixie/InRelease
  403 Forbidden [IP: 13.35.182.118 443]
Error: The repository 'https://download.docker.com/linux/debian trixie InRelease' is
no longer signed.
Notice: Updating from such a repository can't be done securely, and is therefore dis
abled by default.
Notice: See apt-secure(8) manpage for repository creation and user configuration det
ails.
linux-perf is already the newest version (6.12.57-1).
git is already the newest version (1:2.47.3-0+deb13u1).
The following packages were automatically installed and are no longer required:
  bup-doc libmpdec3 par2
  distro-info-data libnsl2 python-apt-common
  libffi7 libostree-1-1 python3.9
  libflatpak0 libpython3.9-minimal python3.9-minimal
  libgdk-pixbuf-xlib-2.0-0 libpython3.9-stdlib tdb-tools
  libgdk-pixbuf2.0-0 libssl1.1
  libmalcontent-ui-1-1 libxdo3
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 164
bagheri-timarchi@debian:~/Desktop/OS-HW7-P-Q3/7-initial$ perf --version
perf version 6.12.57
bagheri-timarchi@debian:~/Desktop/OS-HW7-P-Q3/7-initial$ cat /proc/sys/kernel/perf_e
vent_paranoid
1
bagheri-timarchi@debian:~/Desktop/OS-HW7-P-Q3/7-initial$ sudo sysctl kernel.perf_eve
nt_paranoid=0
kernel.perf_event_paranoid = 0
bagheri-timarchi@debian:~/Desktop/OS-HW7-P-Q3/7-initial$ cat /proc/sys/kernel/perf_e
vent_paranoid
0
bagheri-timarchi@debian:~/Desktop/OS-HW7-P-Q3/7-initial$

```

سپس فایل ها را با دستور مناسب کامپایل می کنیم:

```
bagheri-timarchi@debian:~/Desktop/OS-HW7-P-Q3/7-initial$ gcc -std=c11 -Wall -O2 -g -fno-omit-frame-pointer cli.c fs.c -o myfs
fs.c: In function 'init_fs':
fs.c:121:5: warning: 'strncpy' specified bound 256 equals destination size [-Wstringop-truncation]
   121 |     strncpy(g_fs_path, path, sizeof(g_fs_path));
       |     ^~~~~~
bagheri-timarchi@debian:~/Desktop/OS-HW7-P-Q3/7-initial$ perf record -F 99 -g -- ./myfs
FS CLI - type 'help' for commands.
myfs> stress
--- Starting Stress Test (Linked List Baseline) ---
Created new filesystem 'stress_test.bin'
num_files: 2000
num_operations: 10000
```

در این تمرین 2000 فایل ساخته و در 10,000 مرتبه، بر روی یک فایل رندوم، همه عملیات های زیر را انجام می دهیم:

write, read, shrink, remove, re-create

```
QCALEGWHDH
QDFQHSWZAJ
JSKPQAEQEX
GKZMGGQNL
QZALXZLGTW
DLQDRGJQDQ

Stress Test Finished in: 35.9221 seconds
myfs> exit
Bye!
[ perf record: Woken up 115 times to write data ]
[ perf record: Captured and wrote 29.242 MB perf.data (3666 samples) ]
bagheri-timarchi@debian:~/Desktop/OS-HW7-P-Q3/7-initial$
```

حدود ۳۶ ثانیه اجرای stressTest زمان می برد.

اکنون perf.data را بررسی می کنیم:

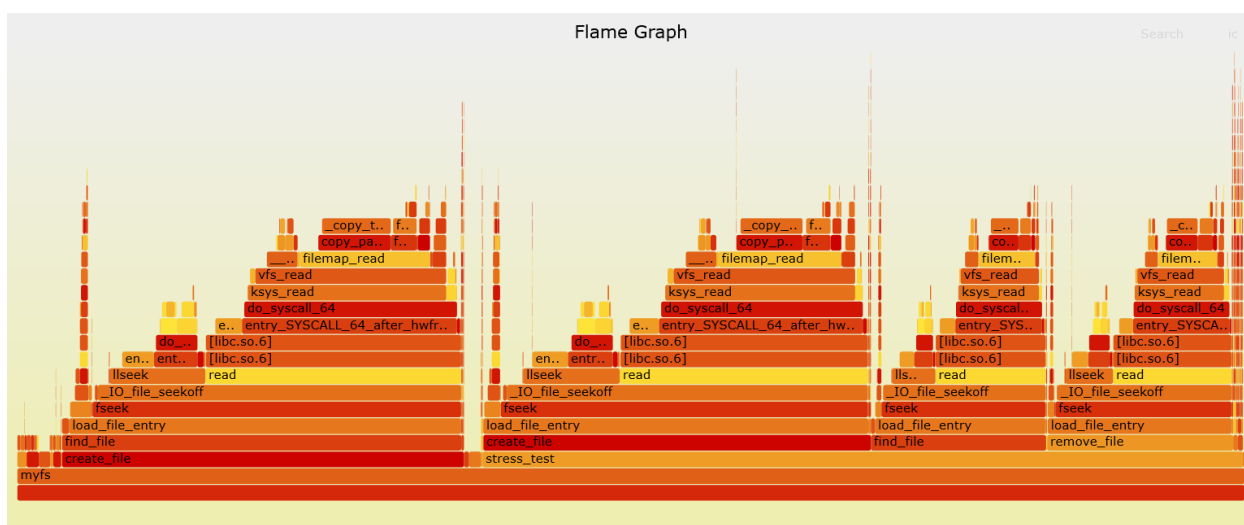
```
part-1 : perf — Konsole
New Tab Split View Copy Paste Find...
Samples: 3K of event 'cpu core/cycles/P', Event count (approx.): 79099125668
Children Self Command Shared Object Symbol
+ 89.08% 0.80% myfs libc.so.6 [.] fseek
+ 88.33% 4.07% myfs libc.so.6 [.] _IO_file_seekoff
+ 65.10% 0.22% myfs myfs [.] create_file
+ 63.43% 1.46% myfs [kernel.kallsyms] [k] entry_SYSCALL_64_after_hwfram
+ 62.18% 2.89% myfs [kernel.kallsyms] [k] do_syscall_64
+ 61.96% 0.00% myfs libc.so.6 [.] 0x00007fce993176ac
+ 61.77% 0.06% myfs libc.so.6 [.] read
+ 61.57% 0.00% myfs myfs [.] stress_test
+ 56.77% 0.00% myfs libc.so.6 [.] 0x00007fce99317687
+ 49.08% 0.19% myfs myfs [.] find_file
+ 48.13% 0.00% myfs myfs [.] load_file_entry (inlined)
+ 47.44% 0.34% myfs [kernel.kallsyms] [k] ksys_read
+ 45.56% 3.65% myfs [kernel.kallsyms] [k] vfs_read
+ 31.28% 0.00% myfs myfs [.] load_file_entry (inlined)
+ 30.94% 4.17% myfs [kernel.kallsyms] [k] filemap_read
+ 24.76% 7.61% myfs libc.so.6 [.] llseek
+ 16.86% 0.82% myfs [kernel.kallsyms] [k] copy_page_to_iter
+ 16.09% 15.98% myfs [kernel.kallsyms] [k] _copy_to_iter
+ 14.21% 0.00% myfs myfs [.] remove_file
+ 14.13% 1.92% myfs [kernel.kallsyms] [k] entry_SYSCALL_64
+ 14.02% 0.00% myfs myfs [.] load_file_entry (inlined)
+ 7.95% 7.83% myfs [kernel.kallsyms] [k] entry_SYSRETQ_unsafe_stack
+ 7.66% 2.64% myfs [kernel.kallsyms] [k] _fsnotify_parent
+ 7.11% 0.72% myfs [kernel.kallsyms] [k] filemap_get_pages
+ 6.44% 1.41% myfs [kernel.kallsyms] [k] syscall_exit_to_user_mode
+ 6.33% 3.81% myfs [kernel.kallsyms] [k] filemap_get_read_batch
+ 4.83% 4.83% myfs libc.so.6 [.] 0x0000000000008f685
+ 4.83% 0.00% myfs libc.so.6 [.] 0x00007fce99317685
+ 4.62% 0.95% myfs libc.so.6 [.] _IO_fread
+ 4.36% 3.61% myfs [kernel.kallsyms] [k] arch_exit_to_user_mode_prepar
+ 3.66% 0.73% myfs [kernel.kallsyms] [k] ksys_lseek
+ 3.21% 3.04% myfs [kernel.kallsyms] [k] fdget_pos
+ 2.58% 0.00% myfs libc.so.6 [.] 0x00007fce993139ef
+ 2.52% 0.00% myfs libc.so.6 [.] _IO_file_underflow
+ 2.49% 0.25% myfs [kernel.kallsyms] [k] touch_atime
+ 2.36% 0.50% myfs [kernel.kallsyms] [k] rw_verify_area
+ 2.24% 0.79% myfs [kernel.kallsyms] [k] atime_needs_update
+ 1.85% 0.45% myfs [kernel.kallsyms] [k] security_file_permission
+ 1.75% 0.34% myfs [kernel.kallsyms] [k] dget_parent
+ 1.72% 0.39% myfs [kernel.kallsyms] [k] dput
Tip: For hierarchical output, try: perf report --hierarchy
```

مشاهده می کنیم که fseek در صدر است.

و flamegraph را می‌سازیم:

```
7-initial : bash — Konsole
New Tab Split View
Copy Paste Find...
bagheri-timarchi@debian:~/Desktop/OS-HW7-P-Q3/7-initial$ git clone https://github.com/brendangregg/FlameGraph.git
Cloning into 'FlameGraph'...
remote: Enumerating objects: 1291, done.
remote: Counting objects: 100% (695/695), done.
remote: Compressing objects: 100% (135/135), done.
remote: Total 1291 (delta 586), reused 560 (delta 560), pack-reused 596 (from 2)
Receiving objects: 100% (1291/1291), 1.92 MiB | 1.44 MiB/s, done.
Resolving deltas: 100% (764/764), done.
bagheri-timarchi@debian:~/Desktop/OS-HW7-P-Q3/7-initial$ perf script | ./FlameGraph/stackcollapse-perf.pl | ./FlameGraph/flamegraph.pl > flamegraph.svg
bagheri-timarchi@debian:~/Desktop/OS-HW7-P-Q3/7-initial$
```

که به صورت زیر است:



محور X تصویر بالا (محور افقی) نشان دهنده زمان مصرف شده cpu برای هر تابع است، محور Y (محور عمودی) نشان دهنده call stack است (ترتیب فراخوانی توابع). با بررسی flamegraph به دنبال توابعی هستیم که در لایه هایی بالایی هستند و عرض زیادی نیز دارند (cpu-time بالا). ۳ سطر بالاتر از stress_test، تابع fseek توجه ما را جلب می‌کند که در هر ۳ بخش create_file, find_file, remove_file وجود دارد و بسیار زمان‌بر است.

علت کاربرد زیاد `fseek` این است که ما برای یافتن فضای خالی و یا بررسی فایل های ایجاد شده، از داده ساختار `linked list` استفاده می کنیم، و این یعنی در تمام این توابع لازم است که مکررا بر روی هر داده از `linked list` حرکت کرده و به تکرار پوینتر فایل را جابه جا کنیم، که برای انجام آن از `fseek` استفاده می کنیم.

برای مثال در `create_file`:

```
uint32_t create_file(const char *name) {
    } else {
        uint32_t cur = sb.file_list_head;
        file_entry_t temp;
        while (1) {
            load_file_entry(cur, &temp);
            if (temp.next == 0) break;
            cur = temp.next;
        }
        temp.next = off;
        save_file_entry(cur, &temp);
    }
}
```

به تعداد دفعات زیادی (حدودا معادل با تعداد فایل ها: 2000) تابع `load_file_entry()` را صدا می کنیم که در آن از `fseek` استفاده شده است:

```
// ----- create file and entry -----
bool load_file_entry(uint32_t offset, file_entry_t *ent) {
    fseek(g_fs_file, offset, SEEK_SET); // set the pointer at proper offset
    size_t n = fread(ent, 1, sizeof(file_entry_t), g_fs_file);
    return (n == sizeof(file_entry_t));
}
```

BitMap

اکنون ساختار مدیریت fs را از linked list به bitmap تغییر می‌دهیم. بلاک اول مربوط به superblock است که اطلاعات کلی fs را شامل می‌شود. بلاک بعدی 32K bit دارد و هر بیت در صورتی صفر است که بلاک متناظر با آن در fs استفاده نشده باشد. با همین منطق، ۲ بیت ابتدایی bitmap همیشه ۱ است. توابع مختلف نیاز به تغییر داشتند. در ادامه

عملکرد جدید دو تابع را بررسی می‌کنیم:

```
// ----- Allocator (Bitmap) -----
uint32_t fs_alloc() {
    uint8_t bitmap[BLOCK_SIZE];

    fseek(g_fs_file, BITMAP_BLOCK_OFFSET, SEEK_SET);
    if (fread(bitmap, 1, BLOCK_SIZE, g_fs_file) != BLOCK_SIZE) return 0;

    // Find first free bit starting from 2 (skip SB and Bitmap)
    for (int i = 2; i < TOTAL_BLOCKS; i++) {
        if (!get_bit(bitmap, i)) {
            // Found free block
            set_bit(bitmap, i);

            // Save updated bitmap
            fseek(g_fs_file, BITMAP_BLOCK_OFFSET, SEEK_SET);
            fwrite(bitmap, 1, BLOCK_SIZE, g_fs_file);
            fflush(g_fs_file);

            // Zero out the newly allocated block
            uint32_t offset = i * BLOCK_SIZE;
            uint8_t zeros[BLOCK_SIZE] = {0};
            fseek(g_fs_file, offset, SEEK_SET);
            fwrite(zeros, 1, BLOCK_SIZE, g_fs_file);

            return offset;
        }
    }

    printf("Error: Filesystem full!\n");
    return 0;
}
```

```
void fs_free(uint32_t offset) {
    if (offset == 0) return;

    uint32_t block_idx = offset / BLOCK_SIZE;
    if (block_idx < 2 || block_idx >= TOTAL_BLOCKS) return;

    uint8_t bitmap[BLOCK_SIZE];
    fseek(g_fs_file, BITMAP_BLOCK_OFFSET, SEEK_SET);
    fread(bitmap, 1, BLOCK_SIZE, g_fs_file);

    clear_bit(bitmap, block_idx);

    fseek(g_fs_file, BITMAP_BLOCK_OFFSET, SEEK_SET);
    fwrite(bitmap, 1, BLOCK_SIZE, g_fs_file);
    fflush(g_fs_file);
}
```

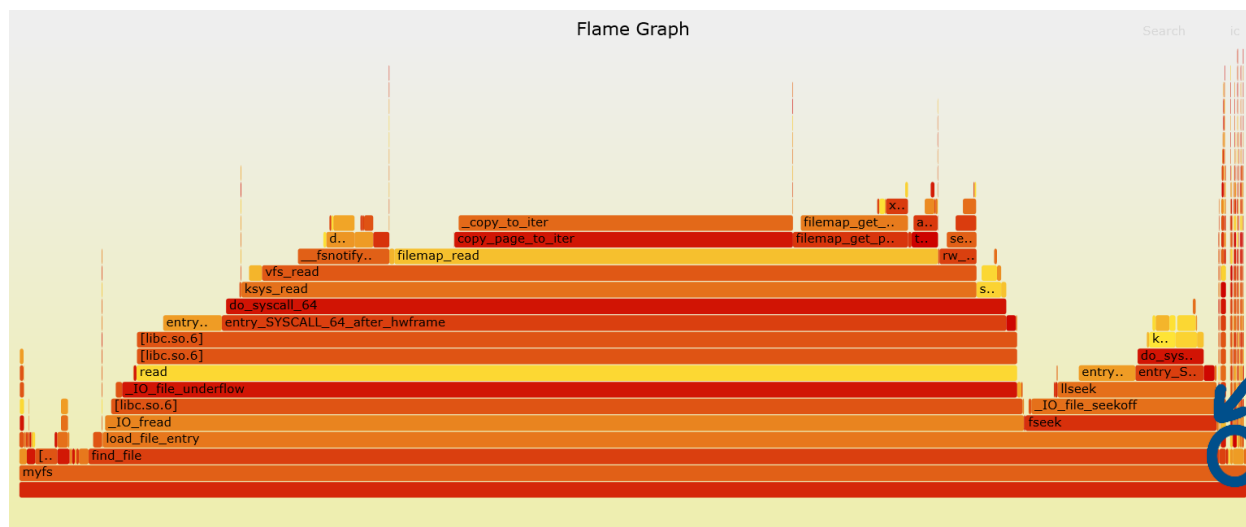
با اجرای نسخه جدید ffs، مشاهده می‌کنیم که stressTest با همان مولفه های تعداد فایل و تعداد عملیات قبل، حدود ۲۵ ثانیه زمان می‌برد که ۱۱ ثانیه کمتر از نسخه قبل و بسیار قابل توجه است.

```
bagheri-timarchi@debian:~/Desktop/second_test_before/part-2$ perf record -F 99 -g --
call-graph dwarf -- ./myfs
FS CLI - type 'help' for commands.
myfs> stress
--- Starting Stress Test (Linked List Baseline) ---
Created new filesystem 'stress test.bin' (128MB, Bitmap Allocator)

LJTVACWRKD
JBYIIAJVHU
TFOMLPCHBN
UKTZINJLOU
PCWRLTAIYT
OIYFKHGZAD
WETJNEZHSQ
RIWKIGKEVB

Stress Test Finished in: 24.9448 seconds
myfs> exit
Bye!
[ perf record: Woken up 84 times to write data ]
[ perf record: Captured and wrote 21.095 MB perf.data (2634 samples) ]
bagheri-timarchi@debian:~/Desktop/second_test_before/part-2$
```

flamegraph این اجرا به شکل زیر است:



همانطور که مشاهده می‌کنید، میزان زمانی که fseek از پردازنده اشغال می‌کند به شدن کاهش یافته و در مقابل IO_fread بسیار کمتر است.

همچنین توابع `remove_file` و `create_file` که در `flamegraph` نسخه قبل زمان قابل توجهی داشتند، در این نسخه

درون دایره آبی رنگ قرار گرفته و بسیار سریع تر شده اند.

